

Beginner Guide to Kubernetes Requests and Limits

Setting CPU and memory boundaries is the single most effective way to keep your applications stable in production. Without these settings, one misbehaving container can consume all server resources and crash everything around it.

What you'll learn:

- 1 Configuring Memory Requests and Limits
- 2 Diagnosing OOMKilled with Exit Code 137
- 3 Handling CPU Requests and Avoiding Throttling
- 4 Monitoring Active Usage with Metrics Server

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Configuring Memory Requests and Limits

Think of requests as reserving a table at a restaurant, while limits are the maximum food you are allowed to order. If a container asks for 256Mi (request), Kubernetes only places it on a node with at least 256Mi free. If the container later tries to cross 512Mi (limit), the Linux kernel immediately sends a kill signal. For memory-sensitive runtimes like Node.js or JVM, keeping limits clear prevents noisy neighbors from degrading your host.

```
$resources:
  requests:
    memory: "256Mi"
  limits:
    memory: "512Mi"
```

Cloud & DevOps Daily

Kubernetes Resource Requests & Limits: Stop Getting OOMKilled

Wednesday, September 23, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Diagnosing OOMKilled with Exit Code 137

When a container consumes more memory than its limit, Kubernetes kills it with Exit Code 137 (128 + 9 for SIGKILL). You will usually notice your pod count showing frequent restarts under 'kubectl get pods'. Running 'kubectl describe pod' lets you inspect the 'Last State' section where 'Reason: OOMKilled' appears plainly. Once spotted, you can increase limits or look into your code for unclosed database connections or memory leaks.

```
$kubectl describe pod my-api-pod | grep -A 5 "Last State:"
```

Cloud & DevOps Daily

Kubernetes Resource Requests & Limits: Stop Getting OOMKilled

Wednesday, September 23, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Handling CPU Requests and Avoiding Throttling

CPU is a compressible resource, meaning Kubernetes will not kill your pod if it needs extra CPU; it simply throttles (slows down) the container. CPU is measured in millicores, where 500m represents half of a single CPU core. Setting realistic CPU requests ensures your container gets enough processor time to handle everyday traffic without starving background jobs.

```
$resources:
  requests:
    cpu: "250m"
  limits:
    cpu: "1000m"
```

Cloud & DevOps Daily

Kubernetes Resource Requests & Limits: Stop Getting OOMKilled

Wednesday, September 23, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Monitoring Active Usage with Metrics Server

Before guessing numbers for requests and limits, check what your pods actually consume during normal traffic. The 'kubectl top' command queries the Kubernetes Metrics Server to report live CPU cores and memory megabytes. Sorting the output by memory highlights which microservice is closest to its limit so you can adjust YAML values before users notice an outage.

```
$kubectl top pods -n default --sort-by=memory
```

Cloud & DevOps Daily

Kubernetes Resource Requests & Limits: Stop Getting OOMKilled

Wednesday, September 23, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always define both requests and limits for every production container.
- + Base your request numbers on real baseline usage observed using 'kubectl top'.
- + Set memory requests equal to memory limits for predictable database and JVM workloads.
- + Be careful with tight CPU limits; throttling often causes unexplained API latency.
- + Check previous logs using '--previous' right after an unexpected pod restart.

Watch Out For

- ! Setting limits without requests, which causes Kubernetes to default requests equal to limits.
- ! Ignoring Exit Code 137 and assuming restarts are caused by network or application bugs.
- ! Setting memory limits lower than the application startup overhead, causing boot crash loops.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh